TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

ĐẠI HỌC QUỐC GIA TP.HCM

BÁO CÁO BÀI TẬP TUẦN

Môn học: Kỹ thuật lập trình (MTH10107)

Giảng viên hướng dẫn: Phạm Thi Vương

Nhóm sinh viên thực hiện:

Phạm Trần Tấn Phát	20280070
Nguyễn Thị Ngọc Anh	23280037
Ngô Văn Phước	24280041

TP.HCM, 2025

Mục lục

1 Tuần 1: File I/O + String cơ bản + Pipeline M0 3

1.1 Đọc và đếm dòng file [Cơ bản] . . . 3
1.1.1 Phân tích & Thiết kế (M0) . . . 3
1.1.2 Mã nguồn C++11 (Tách module) . . . 3
1.1.3 Kế hoạch kiểm thử (Test Cases) . . . 5
1.2 Tách trường CSV 1 hàng [Cơ bản] . . . 5
1.2.1 Phân tích & Thiết kế (M0) . . . 5
1.2.2 Mã nguồn C++11 (Tách module) . . . 6
1.2.3 Kế hoạch kiểm thử (Test Cases) . . . 7
1.3 Bài 3. Phát hiện missing token [Cơ bản] . . . 8
1.3.1 Phân tích & Thiết kế (M0) . . . 8
1.3.2 Mã nguồn C++11 (Tách module) . . . 9
1.3.3 Kế hoạch kiểm thử (Test Cases) . . . 11
1.4 Thống kê cột số từ CSV [Trung bình] . . . 12
1.4.1 Phân tích & Thiết kế (M0) . . . 12
1.4.2 Mã nguồn C++11 (Tách module) . . . 13
1.4.3 Kế hoạch kiểm thử (Test Cases) . . . 16
1.5 Ghi report CSV nhiều cột [Trung bình] . . . 16
1.5.1 Phân tích & Thiết kế (M0) . . . 16
1.5.2 Mã nguồn C++11 (Tách module) . . . 18
1.5.3 Kế hoạch kiểm thử (Test Cases) . . . 22
1.6 Đếm từ trong file text [Cơ bản] . . . 23
1.6.1 Phân tích & Thiết kế (M0) . . . 23
1.6.2 Mã nguồn C++11 (Tách module) . . . 23
1.6.3 Kế hoạch kiểm thử (Test Cases) . . . 25
1.7 Kiểm tra file CSV có header [Cơ bản] . . . 26
1.7.1 Phân tích & Thiết kế (M0) . . . 26
1.7.2 Mã nguồn C++11 (Tách module) . . . 27
1.7.3 Kế hoạch kiểm thử (Test Cases) . . . 29
1.8 Lọc dòng theo điều kiện [Trung bình] . . . 29
1.8.1 Phân tích & Thiết kế (M0) . . . 29
1.8.2 Mã nguồn C++11 (Tách module) . . . 30
1.8.3 Kế hoạch kiểm thử (Test Cases) . . . 33
1.9 Merge 2 file CSV theo id [Trung bình] . . . 34
1.9.1 Phân tích & Thiết kế (M0) . . . 34
1.9.2 Mã nguồn C++11 (Tách module) . . . 36
1.9.3 Kế hoạch kiểm thử (Test Cases) . . . 39

1.10 Phân tích log file [Nâng cao] 41
1.10.1 Phân tích & Thiết kế (M0) 41
1.10.2 Mã nguồn C++11 (Tách module) 42
1.10.3 Kế hoạch kiểm thử (Test Cases) 44

2 Tuần 2: Mảng 1D + Đa thức + Thống kê mô tả 45
2.1 Tìm min, max, mean mảng [Cơ bản] 45
2.1.1 Phân tích & Thiết kế (M0) 45
2.1.2 Mã nguồn C++11 (Tách module) 45
2.1.3 Kế hoạch kiểm thử (Test Cases) 47
2.2 Đếm phần tử thỏa điều kiện [Cơ bản] 48
2.2.1 Phân tích & Thiết kế (M0) 48
2.2.2 Mã nguồn C++11 (Tách module) 49
2.2.3 Kế hoạch kiểm thử (Test Cases) 51
2.3 Tính đa thức tại nhiều điểm [Cơ bản] 51
2.3.1 Phân tích & Thiết kế (M0) 51
2.3.2 Mã nguồn C++11 (Tách module) 52
2.3.3 Kế hoạch kiểm thử (Test Cases) 55
2.4 Variance và std deviation [Trung bình] 56
2.4.1 Phân tích & Thiết kế (M0) 56
2.4.2 Mã nguồn C++11 (Tách module) 57
2.4.3 Kế hoạch kiểm thử (Test Cases) 59
2.5 Đạo hàm đa thức [Trung bình] 59
2.5.1 Phân tích & Thiết kế (M0) 59
2.5.2 Mã nguồn C++11 (Tách module) 60
2.5.3 Kế hoạch kiểm thử (Test Cases) 63
2.6 Tìm vị trí phần tử lớn nhất [Cơ bản] 64
2.6.1 Phân tích & Thiết kế (M0) 64
2.6.2 Mã nguồn C++11 (Tách module) 65
2.6.3 Kế hoạch kiểm thử (Test Cases) 66
2.7 Bài 7: Chuẩn hoá mảng Min-Max [Cơ bản] 67
2.7.1 Phân tích & Thiết kế (M0) 67
2.7.2 Mã nguồn C++11 (Tách module) 68
2.7.3 Kế hoạch kiểm thử (Test Cases) 69
2.8 Impute missing bằng median [Trung bình] 70
2.8.1 Phân tích & Thiết kế (M0) 70
2.8.2 Mã nguồn C++11 (Tách module) 71
2.8.3 Kế hoạch kiểm thử (Test Cases) 74
2.9 Histogram đơn giản [Trung bình] 75
2.9.1 Phân tích & Thiết kế (M0) 75
2.9.2 Mã nguồn C++11 (Tách module) 76
2.9.3 Kế hoạch kiểm thử (Test Cases) 78
2.10 Nội suy tuyến tính [Nâng cao] 79
2.10.1 Phân tích & Thiết kế (M0) 79
2.10.2 Mã nguồn C++11 (Tách module) 80
2.10.3 Kế hoạch kiểm thử (Test Cases) 83

1 Tuần 1: File I/O + String cơ bản + Pipeline M0

1.1 Đọc và đếm dòng file [Cơ bản]

1.1.1 Phân tích & Thiết kế (M0)

- **Input**: File text.txt chứa nhiều dòng (có thể có dòng rỗng, dòng chỉ chứa khoảng trắng)
- **Xử lý**:
  - **Case biên:** File rỗng → `non_empty = 0`, `empty = 0`
  - **Module 1 (Đọc file):**
    - Hàm `readLines(path, vector<string>&)`
    - Đọc từng dòng bằng `getline`
  - **Module 2 (Xử lý):**
    - Trim mỗi dòng (bỏ khoảng trắng đầu/cuối)
    - Nếu sau trim: rỗng → `empty++`; ngược lại → `non_empty++`
  - **Module 3 (Ghi file):** Hàm `writeOutput(path, non_empty, empty)`
- **Output:**

```
non_empty=<n> / empty=<m>
```

1.1.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

// MODULE: Trim string
string trim(const string& s) {
```

```
        size_t start = s.find_first_not_of(" \t\r\n");
        size_t end = s.find_last_not_of(" \t\r\n");

        if (start == string::npos) return "";
        return s.substr(start, end - start + 1);
    }

    // MODULE 1: Read file
    bool readLines(string path, vector<string>& lines) {
        ifstream file(path);
        if (!file.is_open()) return false;

        string line;
        while (getline(file, line)) {
            lines.push_back(line);
        }

        file.close();
        return true;
    }

    // MODULE 2: Count lines
    void countLines(const vector<string>& lines, int& non_empty, int&
    empty) {
        non_empty = 0;
        empty = 0;

        for (const string& line : lines) {
            string t = trim(line);
            if (t.empty()) empty++;
            else non_empty++;
        }
    }

    // MODULE 3: Write output
    bool writeOutput(string path, int non_empty, int empty) {
        ofstream file(path);
        if (!file.is_open()) return false;

        file << "non_empty=" << non_empty
            << " / empty=" << empty;

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<string> lines;
```

```
        int non_empty, empty;

        if (!readLines("text.txt", lines)) {
            cerr << "Khong mo duoc file text.txt\n";
            return 1;
        }

        countLines(lines, non_empty, empty);

        if (!writeOutput("output.txt", non_empty, empty)) {
            cerr << "Khong ghi duoc file output.txt\n";
            return 1;
        }

        return 0;
    }

```

1.1.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (text.txt):

```
Hello

World
```

Expected Output:

```
non_empty=2 / empty=1
```

Test Case 2 (Case biên – file rỗng)
Input (text.txt):

```

```

Expected Output:

```
non_empty=0 / empty=0
```

1.2 Tách trường CSV 1 hàng [Cơ bản]

1.2.1 Phân tích & Thiết kế (M0)

- **Input:** 1 dòng CSV (từ stdin hoặc file), ví dụ:

```
        101, Nguyen Van A, , NA, 88.5

```

- **Xử lý:**
  - **Case biên:**
    - Dòng rỗng → 1 trường rỗng
    - Dòng chỉ chứa dấu phẩy → 2 trường rỗng
  - **Module 1 (Trim):** Xóa khoảng trắng đầu/cuối mỗi field
  - **Module 2 (Split CSV):**
    - Dùng `stringstream + getline(delimiter=',')`
    - Trả về `vector<string>`
  - **Module 3 (Output):**
    - In:
      - số trường
      - từng field trên 1 dòng dạng `'value'`
- **Output:**

```
5
'101'
'Nguyen Van A'
''
'NA'
'88.5'
```

1.2.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <sstream>

using namespace std;

// MODULE: Trim
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: Split CSV
vector<string> splitCSV(const string& line) {
    vector<string> result;

    \begin{minted}{cpp}
```

```
        // Case: line is empty
        \end{minted}
        if (line.empty()) {
            result.push_back("");
            return result;
        }

        stringstream ss(line);
        string token;

        while (getline(ss, token, ',')) {
            result.push_back(trim(token));
        }

        \begin{minted}{cpp}
        // Case: line ends with a comma
        \end{minted}
        if (!line.empty() && line.back() == ',') {
            result.push_back("");
        }

        return result;
    }

    // MODULE: Output
    void printResult(const vector<string>& fields) {
        cout << fields.size() << endl;
        for (const string& f : fields) {
            cout << "'" << f << "'" << endl;
        }
    }

    // MAIN
    int main() {
        string line;
        getline(cin, line);

        vector<string> fields = splitCSV(line);

        printResult(fields);

        return 0;
    }
```

1.2.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường) Input:

```
101, KTLT, , NA, 88.5
```

Expected Output:

```
5
'101'
'KTLT'
''
'NA'
'88.5'
```

Test Case 2 (Case biên – dòng rỗng)
Input:

```

```

Expected Output:

```
1
''
```

Test Case 3 (Case biên – chỉ dấu phẩy)
Input:

```
,
```

Expected Output:

```
2
''
''
```

1.3 Bài 3. Phát hiện missing token [Cơ bản]

1.3.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `data.csv` (1 cột), mỗi dòng là một token
- **Xử lý:**
  - **Case biên:**
    * Token rỗng hoặc chỉ chứa khoảng trắng → missing
    * Token `"na"`, `"nan"`, `"n/a"` (không phân biệt hoa/thường) → missing
    * Token `"0"` → **không phải missing**
    * File rỗng → không có dòng nào bị missing
  - **Module 1 (Chuẩn hóa token):**
    * Hàm `trim(string)`: bỏ khoảng trắng đầu/cuối
    * Hàm `toLowerStr(string)`: chuyển về chữ thường

- **Module 2 (Kiểm tra missing):**
  - Hàm `isMissingToken(string t)`
  - Quy trình:
    - trim → tolower
    - nếu rỗng hoặc thuộc {`na, nan, n/a`} → true
    - ngược lại → false
- **Module 3 (Đọc file & phát hiện):**
  - Hàm `findMissingLines(path, vector<int>&)`
  - Đọc từng dòng bằng `getline`
  - Nếu dòng là missing → lưu chỉ số dòng (0-based)
- **Module 4 (Ghi output):**
  - Hàm `writeOutput(path, indices)`
  - In danh sách chỉ số dòng bị missing

- **Output:**

  Danh sách chỉ số dòng (0-based) bị missing, phân tách bằng khoảng trắng

1.3.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

// MODULE: Trim
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: To lower
string toLowerStr(string s) {
    for (char& c : s) {
        c = (char)tolower((unsigned char)c);
    }
    return s;
}

```

```
// MODULE 1: Check missing token
bool isMissingToken(string t) {
    t = trim(t);
    t = toLowerStr(t);

    if (t.empty()) return true;
    if (t == "na" || t == "nan" || t == "n/a") return true;

    return false;
}

// MODULE 2: Read file & detect missing lines
bool findMissingLines(string path, vector<int>& indices) {
    ifstream file(path);
    if (!file.is_open()) return false;

    string line;
    int idx = 0;

    while (getline(file, line)) {
        if (isMissingToken(line)) {
            indices.push_back(idx);
        }
        idx++;
    }

    file.close();
    return true;
}

// MODULE 3: Write output
bool writeOutput(string path, const vector<int>& indices) {
    ofstream file(path);
    if (!file.is_open()) return false;

    for (size_t i = 1; i < indices.size(); i++) {
        file << indices[i]+1;
        if (i != indices.size() - 1) file << " ";
    }

    file.close();
    return true;
}

// MAIN
int main() {
    vector<int> indices;

    if (!findMissingLines("data.csv", indices)) {
```

```
            cerr << "Khong mo duoc file data.csv\n";
            return 1;
        }

        if (!writeOutput("output.txt", indices)) {
            cerr << "Khong ghi duoc file output.txt\n";
            return 1;
        }

        return 0;
    }
```

1.3.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (`data.csv`):

```
10
NA
hello
nan
N/A
0
```

Expected Output:

```
2 4 5
```

Test Case 2 (Không có missing)
Input (`data.csv`):

```
1
2
3
abc
0
```

Expected Output:

```
```

Test Case 3 (Case biên – file rỗng)
Input (`data.csv`):

```
```

Expected Output:

```
```

1.4 Thống kê cột số từ CSV [Trung bình]

1.4.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `data.csv` gồm 1 cột số thực
  - Có thể có missing; missing được hiểu là ô trống hoặc `NA`/`NaN` (không phân biệt hoa/thường)
- **Xử lý:**
  - **Case biên:**
    * Toàn bộ dòng đều missing → `count = 0`, `missing = số dòng`, và `min/max/mean = N/A`
    * File rỗng → `count = 0`, `missing = 0`, và `min/max/mean = N/A`
    * Chỉ có 1 giá trị hợp lệ ($n = 1$) → `min = max = mean = giá trị đó`
  - **Module 1 (Chuẩn hóa token):**
    * Hàm `trim(string)`: bỏ khoảng trắng đầu/cuối
    * Hàm `toLowerStr(string)`: chuyển về chữ thường
    * Hàm `isMissingToken(string)`: kiểm tra token có phải missing hay không
  - **Module 2 (Đọc CSV 1 cột):**
    * Hàm `readCSV1Col(path, vector<double>&, int& missing)`
    * Đọc từng dòng bằng `getline`
    * Nếu dòng là missing → tăng `missing`
    * Nếu parse được thành số thực → đưa vào vector hợp lệ
    * Nếu token không phải missing nhưng không parse được thành số → bỏ qua để chương trình chạy an toàn
  - **Module 3 (Tính thống kê):**
    * Hàm `computeStats(vector<double>)`
    * Tính:
      · `count`: số giá trị hợp lệ
      · `min`: giá trị nhỏ nhất
      · `max`: giá trị lớn nhất
      · `mean`: trung bình cộng
    * Dùng `long double` để tích lũy tổng, tránh mất độ chính xác
  - **Module 4 (Ghi output):**
    * Hàm `writeOutput(path, stats, missing)`
    * Nếu `count = 0` → ghi `N/A` cho `min/max/mean`

* Nếu `count > 0` → ghi số thực với `fixed << setprecision(2)`

- **Output:**

```
count=<n> / missing=<m> / min=<f> / max=<f> / mean=<f>
```

hoặc nếu không có giá trị hợp lệ:

```
count=0 / missing=<m> / min=N/A / max=N/A / mean=N/A
```

1.4.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <iomanip>

using namespace std;

struct Stats {
    int count;
    double minVal;
    double maxVal;
    double mean;
};

// MODULE: Trim
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: To lower
string toLowerStr(string s) {
    for (char& c : s) {
        if ('A' <= c && c <= 'Z') c = c - 'A' + 'a';
    }
    return s;
}

// MODULE: Check missing token
bool isMissingToken(string t) {
    t = toLowerStr(trim(t));
    return t.empty() || t == "na" || t == "nan" || t == "n/a";
}
```

```

// MODULE: Parse double safely
bool parseDouble(const string& s, double& value) {
    string t = trim(s);
    if (t.empty()) return false;

    try {
        size_t idx;
        value = stod(t, &idx);
        return idx == t.size();
    } catch (...) {
        return false;
    }
}

// MODULE 1: Read CSV 1 column
bool readCSV1Col(string path, vector<double>& data, int& missing) {
    ifstream file(path);
    if (!file.is_open()) return false;

    missing = 0;
    string line;

    while (getline(file, line)) {
        if (isMissingToken(line)) {
            missing++;
        } else {
            double value;
            if (parseDouble(line, value)) {
                data.push_back(value);
            }
        }
    }

    file.close();
    return true;
}

// MODULE 2: Compute stats
Stats computeStats(const vector<double>& data) {
    Stats st;
    st.count = (int)data.size();
    st.minVal = 0.0;
    st.maxVal = 0.0;
    st.mean = 0.0;

    if (data.empty()) return st;

    st.minVal = data[0];
```

```
        st.maxVal = data[0];

        long double sum = 0.0;
        for (double x : data) {
            if (x < st.minVal) st.minVal = x;
            if (x > st.maxVal) st.maxVal = x;
            sum += x;
        }

        st.mean = (double)(sum / data.size());
        return st;
    }

    // MODULE 3: Write output
    bool writeOutput(string path, const Stats& st, int missing) {
        ofstream file(path);
        if (!file.is_open()) return false;

        file << "count=" << st.count
            << " / missing=" << missing
            << " / ";

        if (st.count == 0) {
            file << "min=N/A / max=N/A / mean=N/A";
        } else {
            file << fixed << setprecision(2);
            file << "min=" << st.minVal
                << " / max=" << st.maxVal
                << " / mean=" << st.mean;
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<double> data;
        int missing = 0;

        if (!readCSV1Col("data.csv", data, missing)) {
            cerr << "Khong mo duoc file data.csv\n";
            return 1;
        }

        Stats st = computeStats(data);

        if (!writeOutput("output.txt", st, missing)) {
            cerr << "Khong ghi duoc file output.txt\n";
```

```
            return 1;
        }

        return 0;
    }
```

1.4.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (`data.csv`):

```
10
20.5
NA
30
NaN
15.5
```

Expected Output:

```
count=4 / missing=2 / min=10.00 / max=30.00 / mean=19.00
```

Test Case 2 (Case biên – toàn missing)
Input (`data.csv`):

```
NA

nan
NA
```

Expected Output:

```
count=0 / missing=4 / min=N/A / max=N/A / mean=N/A
```

Test Case 3 (Case biên – chỉ có 1 giá trị hợp lệ)
Input (`data.csv`):

```
NA
42.5
NaN
```

Expected Output:

```
count=1 / missing=2 / min=42.50 / max=42.50 / mean=42.50
```

1.5 Ghi report CSV nhiều cột [Trung bình]

1.5.1 Phân tích & Thiết kế (M0)

- **Input:**

- File `scores.csv`, mỗi dòng gồm 4 cột:

  `id, math, lit, sci`

- Các cột điểm có thể missing
- Missing được hiểu là ô trống hoặc `NA`/`NaN` (không phân biệt hoa/thường)

- **Xử lý:**
  - **Case biên:**
    * Missing môn → coi là `0` khi tính tổng, nhưng vẫn đánh dấu trong record
    * Tất cả môn đều missing → `total = 0` → xếp loại `Yeu`
    * `id` rỗng → ghi `UNKNOWN`
    * Dòng không đủ 4 cột → bỏ qua để chương trình chạy an toàn
    * File rỗng → `result.csv` chỉ có header và dòng summary với tất cả count bằng 0
  - **Module 1 (Chuẩn hóa token):**
    * Hàm `trim(string)`: bỏ khoảng trắng đầu/cuối
    * Hàm `toLowerStr(string)`: chuyển về chữ thường
    * Hàm `isMissingToken(string)`: kiểm tra missing token
  - **Module 2 (Tách & parse 1 dòng CSV):**
    * Hàm `splitCSV(line, vector<string>&)`: tách dòng theo dấu phẩy
    * Hàm `parseScore(cell, value, isMissing)`:
      · nếu missing → gán `value = 0`, `isMissing = true`
      · nếu parse được số → giữ giá trị thật
      · nếu token không hợp lệ → xem như dòng lỗi
    * Hàm `parseRow(cells, Record& rec)` tạo ra một record hoàn chỉnh
  - **Module 3 (Tính tổng & xếp loại):**
    * Tổng:

      `total = math + lit + sci`

    * Hàm `computeRank(total)`:
      · `total` $\geq$ `24` → `Gioi`
      · `total` $\geq$ `18` → `Kha`
      · `total` $\geq$ `12` → `Trung binh`
      · `total` $<$ `12` → `Yeu`
  - **Module 4 (Đọc toàn bộ file):**
    * Hàm `readScores(path, vector<Record>&)`
    * Đọc từng dòng bằng `getline`
    * Với mỗi dòng hợp lệ:
      · parse record

        - tính tổng
        - gán rank
        - đưa vào danh sách kết quả
    - **Module 5 (Ghi report & summary):**
        * Hàm `writeResult(path, records)`
        * Ghi file `result.csv` gồm các cột:

`id, total, rank`

        * Cuối file ghi thêm 1 dòng summary đếm số record theo từng xếp loại
        * Số thực dùng `fixed << setprecision(2)`

- **Output:**
    - File `result.csv`
    - Mỗi dòng dữ liệu có dạng:

`id,total,rank`

    - Dòng cuối là summary:

`SUMMARY,count_gioi,count_kha,count_trung_binh,count_yeu`

1.5.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <iomanip>
#include <cctype>

using namespace std;

struct Record {
    string id;
    double math;
    double lit;
    double sci;
    bool missingMath;
    bool missingLit;
    bool missingSci;
    double total;
    string rank;
};

// MODULE: Trim
```

```
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: To lower
string toLowerStr(string s) {
    for (char& c : s) {
        c = (char)tolower((unsigned char)c);
    }
    return s;
}

// MODULE: Check missing token
bool isMissingToken(string t) {
    t = toLowerStr(trim(t));
    return t.empty() || t == "na" || t == "nan";
}

// MODULE: Parse double safely
bool parseDouble(const string& s, double& value) {
    string t = trim(s);
    if (t.empty()) return false;

    try {
        size_t idx;
        value = stod(t, &idx);
        return idx == t.size();
    } catch (...) {
        return false;
    }
}

// MODULE 1: Split CSV
void splitCSV(const string& line, vector<string>& cells) {
    cells.clear();
    stringstream ss(line);
    string cell;

    while (getline(ss, cell, ',')) {
        cells.push_back(cell);
    }
}

// MODULE 2: Parse one score cell
bool parseScore(const string& cell, double& value, bool& isMissing) {
```

```
        if (isMissingToken(cell)) {
            value = 0.0;
            isMissing = true;
            return true;
        }

        isMissing = false;
        return parseDouble(cell, value);
    }

    // MODULE 3: Compute rank
    string computeRank(double total) {
        if (total >= 24.0) return "Gioi";
        if (total >= 18.0) return "Kha";
        if (total >= 12.0) return "Trung binh";
        return "Yeu";
    }

    // MODULE 4: Parse one row -> Record
    bool parseRow(const vector<string>& cells, Record& rec) {
        if (cells.size() != 4) return false;

        rec.id = trim(cells[0]);
        if (rec.id.empty()) rec.id = "UNKNOWN";

        if (!parseScore(cells[1], rec.math, rec.missingMath)) return false;
        if (!parseScore(cells[2], rec.lit, rec.missingLit)) return false;
        if (!parseScore(cells[3], rec.sci, rec.missingSci)) return false;

        rec.total = rec.math + rec.lit + rec.sci;
        rec.rank = computeRank(rec.total);

        return true;
    }

    // MODULE 5: Read whole file
    bool readScores(string path, vector<Record>& records) {
        ifstream file(path);
        if (!file.is_open()) return false;

        string line;
        while (getline(file, line)) {
            if (trim(line).empty()) continue;

            vector<string> cells;
            splitCSV(line, cells);

            Record rec;
```

```
            if (parseRow(cells, rec)) {
                records.push_back(rec);
            }
        }

        file.close();
        return true;
    }

    // MODULE 6: Write result.csv
    bool writeResult(string path, const vector<Record>& records) {
        ofstream file(path);
        if (!file.is_open()) return false;

        int countGioi = 0;
        int countKha = 0;
        int countTrungBinh = 0;
        int countYeu = 0;

        file << "id,total,rank\n";
        file << fixed << setprecision(2);

        for (size_t i = 0; i < records.size(); i++) {
            file << records[i].id << ","
                << records[i].total << ","
                << records[i].rank << "\n";

            if (records[i].rank == "Gioi") countGioi++;
            else if (records[i].rank == "Kha") countKha++;
            else if (records[i].rank == "Trung binh") countTrungBinh++;
            else countYeu++;
        }

        file << "SUMMARY,"
            << countGioi << ","
            << countKha << ","
            << countTrungBinh << ","
            << countYeu;

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<Record> records;

        if (!readScores("scores.csv", records)) {
            cerr << "Khong mo duoc file scores.csv\n";
```

```
            return 1;
        }

        if (!writeResult("result.csv", records)) {
            cerr << "Khong ghi duoc file result.csv\n";
            return 1;
        }

        return 0;
    }
```

1.5.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (`scores.csv`):

```
SV01,8,9,8
SV02,6,6,7
SV03,4,5,4
```

Expected Output (`result.csv`):

```
id,total,rank
SV01,25.00,Gioi
SV02,19.00,Kha
SV03,13.00,Trung binh
SUMMARY,1,1,1,0
```

Test Case 2 (Có missing)
Input (`scores.csv`):

```
SV01,8,NA,8
SV02,,6,7
,NA,NaN,
```

Expected Output (`result.csv`):

```
id,total,rank
SV01,16.00,Trung binh
SV02,13.00,Trung binh
UNKNOWN,0.00,Yeu
SUMMARY,0,0,2,1
```

Test Case 3 (Case biên – file rỗng)
Input (`scores.csv`):

```

```

Expected Output (`result.csv`):

```
id,total,rank
SUMMARY,0,0,0,0
```

1.6 Đếm từ trong file text [Cơ bản]

1.6.1 Phân tích & Thiết kế (M0)

- **Input:** File `words.txt` gồm nhiều dòng văn bản
- **Xử lý:**
  - **Case biên:**
    - File rỗng → `total_words = 0`, `unique_words = 0`
    - Dòng rỗng hoặc chỉ chứa khoảng trắng → không tạo ra từ nào
  - **Module 1 (Đọc file):**
    - Hàm `readLines(path, vector<string>&)`
    - Đọc toàn bộ file bằng `getline`
  - **Module 2 (Tách từ):**
    - Hàm `splitWords(line, vector<string>&)`
    - Dùng `stringstream` để tách từ theo khoảng trắng
    - Không normalize: không lowercase, không bỏ dấu câu
  - **Module 3 (Đếm từ):**
    - `total_words`: tổng số từ tách được từ toàn bộ file
    - `unique_words`: số từ phân biệt
    - Dùng `set<string>` để lưu các từ duy nhất
  - **Module 4 (Ghi file):**
    - Hàm `writeOutput(path, total_words, unique_words)`
- **Output:**

```
total_words=<n> / unique_words=<m>
```

1.6.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include <set>

using namespace std;

// MODULE 1: Read file
bool readLines(string path, vector<string>& lines) {
    ifstream file(path);
    if (!file.is_open()) return false;
```

```

        string line;
        while (getline(file, line)) {
            lines.push_back(line);
        }

        file.close();
        return true;
    }

    // MODULE 2: Split words by whitespace
    void splitWords(const string& line, vector<string>& words) {
        stringstream ss(line);
        string word;

        while (ss >> word) {
            words.push_back(word);
        }
    }

    // MODULE 3: Count total and unique words
    void countWords(const vector<string>& lines, int& total_words, int&
    unique_words) {
        total_words = 0;
        unique_words = 0;

        set<string> distinct;
        vector<string> words;

        for (const string& line : lines) {
            words.clear();
            splitWords(line, words);

            for (const string& word : words) {
                total_words++;
                distinct.insert(word);
            }
        }

        unique_words = (int)distinct.size();
    }

    // MODULE 4: Write output
    bool writeOutput(string path, int total_words, int unique_words) {
        ofstream file(path);
        if (!file.is_open()) return false;

        file << "total_words=" << total_words
            << " / unique_words=" << unique_words;
```

```

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<string> lines;
        int total_words, unique_words;

        if (!readLines("words.txt", lines)) {
            cerr << "Khong mo duoc file words.txt\n";
            return 1;
        }

        countWords(lines, total_words, unique_words);

        if (!writeOutput("output.txt", total_words, unique_words)) {
            cerr << "Khong ghi duoc file output.txt\n";
            return 1;
        }

        return 0;
    }
```

1.6.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (`words.txt`):

```
hello world
world !
```

Expected Output:

```
total_words=4 / unique_words=3
```

Test Case 2 (Case biên – file rỗng)
Input (`words.txt`):

```

```

Expected Output:

```
total_words=0 / unique_words=0
```

Test Case 3 (Dòng rỗng và nhiều khoảng trắng)
Input (`words.txt`):

```
apple   banana

```

```
   apple
banana
```

Expected Output:

```
total_words=4 / unique_words=2
```

1.7 Kiểm tra file CSV có header [Cơ bản]

1.7.1 Phân tích & Thiết kế (M0)

- **Input:** File CSV bất kỳ (ví dụ: `data.csv`), mỗi dòng gồm nhiều cột phân tách bởi dấu phẩy
- **Xử lý:**
  - **Case biên:**
    - File rỗng → `has_header=0`
    - Dòng đầu rỗng → `has_header=0`
  - **Module 1 (Đọc dòng đầu):**
    - Hàm `readFirstLine(path, string&)`
    - Đọc dòng đầu tiên bằng `getline`
  - **Module 2 (Tách CSV):**
    - Hàm `splitCSV(line, vector<string>&)`
    - Tách theo dấu phẩy
  - **Module 3 (Kiểm tra số):**
    - Hàm `isNumber(string)`
    - Nếu **tất cả cell không parse được thành số** → header
  - **Module 4 (Xác định header):**
    - Nếu mọi cell đều **không phải số** → `has_header=1`
    - Ngược lại → `has_header=0`
  - **Module 5 (Ghi output):**
    - Hàm `writeOutput(path, has_header, columns)`
    - Nếu có header → in thêm danh sách cột
- **Output:**

  `has_header=1` hoặc `has_header=0`

  Nếu có header:

  `columns: col1, col2, ...`

1.7.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>

using namespace std;

// MODULE: Trim
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: Check number
bool isNumber(const string& s) {
    if (s.empty()) return false;

    try {
        size_t idx;
        stod(s, &idx);
        return idx == s.size();
    } catch (...) {
        return false;
    }
}

// MODULE 1: Read first line
bool readFirstLine(string path, string& line) {
    ifstream file(path);
    if (!file.is_open()) return false;

    if (!getline(file, line)) {
        file.close();
        return true; // file rong
    }

    file.close();
    return true;
}

// MODULE 2: Split CSV
void splitCSV(const string& line, vector<string>& cells) {
    stringstream ss(line);
```

```
        string temp;

        while (getline(ss, temp, ',')) {
            cells.push_back(trim(temp));
        }
    }

    // MODULE 3: Detect header
    bool detectHeader(const vector<string>& cells) {
        if (cells.empty()) return false;

        for (const string& c : cells) {
            if (isNumber(c)) return false;
        }
        return true;
    }

    // MODULE 4: Write output
    bool writeOutput(string path, bool has_header, const vector<string>&
    cols) {
        ofstream file(path);
        if (!file.is_open()) return false;

        file << "has_header=" << (has_header ? 1 : 0);

        if (has_header) {
            file << "\ncolumns: ";
            for (size_t i = 0; i < cols.size(); i++) {
                file << cols[i];
                if (i != cols.size() - 1) file << ", ";
            }
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        string line;
        vector<string> cells;

        if (!readFirstLine("data.csv", line)) {
            cerr << "Khong mo duoc file data.csv\n";
            return 1;
        }

        if (!line.empty()) {
            splitCSV(line, cells);
```

```
        }

        bool has_header = detectHeader(cells);

        if (!writeOutput("output.txt", has_header, cells)) {
            cerr << "Khong ghi duoc file output.txt\n";
            return 1;
        }

        return 0;
    }
```

1.7.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường – có header)
Input (data.csv):

```
name, age, score
A, 20, 8.5
```

Expected Output:

```
has_header=1
columns: name, age, score
```

Test Case 2 (Không có header)
Input (data.csv):

```
1, 2, 3
4, 5, 6
```

Expected Output:

```
has_header=0
```

Test Case 3 (Case biên – file rỗng)
Input (data.csv):

```

```

Expected Output:

```
has_header=0
```

1.8 Lọc dòng theo điều kiện [Trung bình]

1.8.1 Phân tích & Thiết kế (M0)

- **Input:**

  - File `data.csv`: mỗi dòng dạng `id, value`
  - File `config.txt`: chứa 2 số thực `lo` và `hi`

- **Xử lý:**
  - **Case biên:**
    * Value missing (rỗng, `NA`/`NaN`) → bỏ (dropped)
    * Value không parse được → bỏ
    * `lo > hi` → không có dòng nào được giữ
    * File rỗng → output rỗng, `kept=0`, `dropped=0`
  - **Module 1 (Chuẩn hóa token):**
    * `trim`, `toLowerStr`, `isMissingToken`
  - **Module 2 (Đọc config):**
    * Hàm `readConfig(path, lo, hi)`
    * Parse 2 số thực từ file
  - **Module 3 (Parse CSV):**
    * `splitCSV(line, cells)`
    * Mỗi dòng phải có đúng 2 cột
  - **Module 4 (Lọc dữ liệu):**
    * Nếu value hợp lệ và `lo` $\leq$ `value` $\leq$ `hi` → kept
    * Ngược lại → dropped
  - **Module 5 (Ghi file):**
    * Ghi `filtered.csv`: các dòng được giữ
    * Cuối file ghi: `kept=<n> / dropped=<m>`

- **Output:**
  - File `filtered.csv`
  - Dòng cuối:

```
kept=<n> / dropped=<m>
```

1.8.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>

using namespace std;

// MODULE: Trim
```

```
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: To lower
string toLowerStr(string s) {
    for (char& c : s) {
        c = (char)tolower((unsigned char)c);
    }
    return s;
}

// MODULE: Check missing
bool isMissingToken(string t) {
    t = toLowerStr(trim(t));
    return t.empty() || t == "na" || t == "nan";
}

// MODULE: Parse double
bool parseDouble(const string& s, double& value) {
    string t = trim(s);
    if (t.empty()) return false;

    try {
        size_t idx;
        value = stod(t, &idx);
        return idx == t.size();
    } catch (...) {
        return false;
    }
}

// MODULE 1: Read config
bool readConfig(string path, double& lo, double& hi) {
    ifstream file(path);
    if (!file.is_open()) return false;

    file >> lo >> hi;

    file.close();
    return true;
}

// MODULE 2: Split CSV
void splitCSV(const string& line, vector<string>& cells) {
```

```
        cells.clear();
        stringstream ss(line);
        string cell;

        while (getline(ss, cell, ',')) {
            cells.push_back(cell);
        }
    }

    // MODULE 3: Filter data
    bool filterData(string inPath, string outPath, double lo, double hi,
                    int& kept, int& dropped) {

        ifstream inFile(inPath);
        if (!inFile.is_open()) return false;

        ofstream outFile(outPath);
        if (!outFile.is_open()) return false;

        kept = 0;
        dropped = 0;

        string line;
        while (getline(inFile, line)) {
            if (trim(line).empty()) continue;

            vector<string> cells;
            splitCSV(line, cells);

            if (cells.size() != 2) {
                dropped++;
                continue;
            }

            string id = trim(cells[0]);
            string valStr = cells[1];

            if (isMissingToken(valStr)) {
                dropped++;
                continue;
            }

            double value;
            if (!parseDouble(valStr, value)) {
                dropped++;
                continue;
            }

            if (value >= lo && value <= hi) {
```

```
                outFile << id << "," << value << "\n";
                kept++;
            } else {
                dropped++;
            }
        }

        outFile << "kept=" << kept << " / dropped=" << dropped;

        inFile.close();
        outFile.close();
        return true;
    }

    // MAIN
    int main() {
        double lo, hi;
        int kept, dropped;

        if (!readConfig("config.txt", lo, hi)) {
            cerr << "Khong mo duoc file config.txt\n";
            return 1;
        }

        if (!filterData("data.csv", "filtered.csv", lo, hi, kept, dropped) ) {
            cerr << "Loi xu ly file\n";
            return 1;
        }

        return 0;
    }
```

1.8.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)

Input (`config.txt`):

```
10 20
```

Input (`data.csv`):

```
A,5
B,10
C,15
D,25
```

Expected Output (`filtered.csv`):

```
B,10
C,15
```

```
kept=2 / dropped=2
```

Test Case 2 (Có missing)
Input (`config.txt`):

```
0 100
```

Input (`data.csv`):

```
A,NA
B,
C,50
D,abc
```

Expected Output:

```
C,50
kept=1 / dropped=3
```

Test Case 3 (Case biên – file rỗng)
Input (`data.csv`):

```

```

Expected Output:

```
kept=0 / dropped=0
```

1.9 Merge 2 file CSV theo id [Trung bình]

1.9.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `fileA.csv`: mỗi dòng có dạng `id, score_A`
  - File `fileB.csv`: mỗi dòng có dạng `id, score_B`
- **Xử lý:**
  - **Case biên:**
    * Một trong hai file rỗng → không join được dòng nào
    * Có id chỉ xuất hiện ở đúng một file → không ghi vào `merged.csv`, nhưng vẫn tính vào thống kê `id_only_one`
    * Missing được hiểu là ô trống hoặc `NA`/`NaN` (không phân biệt hoa/thường); các dòng thiếu dữ liệu hợp lệ sẽ bị bỏ qua
  - **Module 1 (Chuẩn hóa token):**
    * Hàm `trim(string)` để bỏ khoảng trắng đầu/cuối
    * Hàm `toLowerStr(string)` để đưa token về chữ thường khi kiểm tra missing

    * Hàm `isMissingToken(string)` để nhận diện ô trống, `NA`, `NaN`
  - **Module 2 (Parse 1 dòng CSV):**
    * Hàm `parseLine(line, id, score)`
    * Tách dòng theo dấu phẩy thành 2 cột: `id`, `score`
    * Nếu thiếu cột, thiếu dữ liệu, hoặc `score` không parse được thành số → dòng không hợp lệ
  - **Module 3 (Đọc file CSV vào map):**
    * Hàm `readCSV(path, map<string, double>&)`
    * Đọc từng dòng bằng `getline`
    * Với mỗi dòng hợp lệ, lưu `id` → `score`
    * Nếu trùng id trong cùng một file thì giữ giá trị xuất hiện sau cùng
  - **Module 4 (Inner Join theo id):**
    * Duyệt các id của file A
    * Nếu id cũng có trong file B → đưa vào danh sách kết quả join
    * Tính `diff = score_A - score_B`
  - **Module 5 (Đếm thống kê):**
    * `joined_rows`: số id xuất hiện ở cả hai file
    * `id_only_one`: số id chỉ xuất hiện ở đúng một file
    * Công thức:
      $$\texttt{id_only_one} = |\texttt{A only}| + |\texttt{B only}|$$
  - **Module 6 (Ghi file + in báo cáo):**
    * Hàm `writeMerged(path, rows)` ghi file `merged.csv`
    * Định dạng:
      ```
      id, score_A, score_B, diff
      ```
    * Dùng `fixed << setprecision(6)` cho số thực
    * In ra màn hình:
      ```
      joined_rows=<n> / id_only_one=<m>
      ```

- **Output:**
  - File `merged.csv` chứa các dòng inner join:
    ```
    id,score_A,score_B,diff
    ```
  - Màn hình in:
    ```
    joined_rows=<n> / id_only_one=<m>
    ```

1.9.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <map>
#include <iomanip>
#include <cctype>

using namespace std;

struct JoinRow {
    string id;
    double scoreA;
    double scoreB;
    double diff;
};

// MODULE: Trim string
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: To lower
string toLowerStr(string s) {
    for (char& c : s) {
        c = (char)tolower((unsigned char)c);
    }
    return s;
}

// MODULE: Check missing token
bool isMissingToken(const string& s) {
    string t = toLowerStr(trim(s));
    return t.empty() || t == "na" || t == "nan";
}

// MODULE: Parse double safely
bool parseDouble(const string& s, double& value) {
    string t = trim(s);
    if (t.empty()) return false;

    try {
```

```
            size_t idx;
            value = stod(t, &idx);
            return idx == t.size();
        } catch (...) {
            return false;
        }
    }

    // MODULE 1: Parse one CSV line -> id, score
    bool parseLine(const string& line, string& id, double& score) {
        stringstream ss(line);
        string cell1, cell2, extra;

        if (!getline(ss, cell1, ',')) return false;
        if (!getline(ss, cell2, ',')) return false;

        // Neu con du cot thi dong khong dung format id,score
        if (getline(ss, extra, ',')) return false;

        id = trim(cell1);
        string scoreToken = trim(cell2);

        if (isMissingToken(id) || isMissingToken(scoreToken)) return false;
        if (!parseDouble(scoreToken, score)) return false;

        return true;
    }

    // MODULE 2: Read CSV into map<id, score>
    bool readCSV(string path, map<string, double>& data) {
        ifstream file(path);
        if (!file.is_open()) return false;

        string line;
        while (getline(file, line)) {
            line = trim(line);
            if (line.empty()) continue;

            string id;
            double score;

            if (parseLine(line, id, score)) {
                data[id] = score; // Neu trung id, giu gia tri xuat hien sau
            }
        }

        file.close();
```

```
        return true;
    }

    // MODULE 3: Inner join by id
    void innerJoin(const map<string, double>& a,
                const map<string, double>& b,
                vector<JoinRow>& rows,
                int& joined_rows,
                int& id_only_one) {
        joined_rows = 0;
        id_only_one = 0;

        for (map<string, double>::const_iterator it = a.begin(); it != a.end(); ++it) {
            map<string, double>::const_iterator found = b.find(it->first);
            if (found != b.end()) {
                JoinRow row;
                row.id = it->first;
                row.scoreA = it->second;
                row.scoreB = found->second;
                row.diff = row.scoreA - row.scoreB;
                rows.push_back(row);
                joined_rows++;
            } else {
                id_only_one++;
            }
        }

        for (map<string, double>::const_iterator it = b.begin(); it != b.end(); ++it) {
            if (a.find(it->first) == a.end()) {
                id_only_one++;
            }
        }
    }

    // MODULE 4: Write merged.csv
    bool writeMerged(string path, const vector<JoinRow>& rows) {
        ofstream file(path);
        if (!file.is_open()) return false;

        file << "id,score_A,score_B,diff\n";
        file << fixed << setprecision(6);

        for (size_t i = 0; i < rows.size(); i++) {
            file << rows[i].id << ","
                << rows[i].scoreA << ","
                << rows[i].scoreB << ","
                << rows[i].diff << "\n";
```

```
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        map<string, double> fileAData;
        map<string, double> fileBData;
        vector<JoinRow> rows;
        int joined_rows = 0;
        int id_only_one = 0;

        if (!readCSV("fileA.csv", fileAData)) {
            cerr << "Khong mo duoc file fileA.csv\n";
            return 1;
        }

        if (!readCSV("fileB.csv", fileBData)) {
            cerr << "Khong mo duoc file fileB.csv\n";
            return 1;
        }

        innerJoin(fileAData, fileBData, rows, joined_rows, id_only_one);

        if (!writeMerged("merged.csv", rows)) {
            cerr << "Khong ghi duoc file merged.csv\n";
            return 1;
        }

        cout << "joined_rows=" << joined_rows
            << " / id_only_one=" << id_only_one << "\n";

        return 0;
    }
```

1.9.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (fileA.csv):

```
A01,8.5
A02,7.0
A03,9.0
```

Input (fileB.csv):

```
A02,6.5
A03,9.5
```

```
A04,8.0
```

Expected Output (merged.csv):

```
id,score_A,score_B,diff
A02,7.000000,6.500000,0.500000
A03,9.000000,9.500000,-0.500000
```

Expected Console Output:

```
joined_rows=2 / id_only_one=2
```

Test Case 2 (Case biên – một file rỗng)
Input (fileA.csv):

```

```

Input (fileB.csv):

```
B01,5.0
B02,6.0
```

Expected Output (merged.csv):

```
id,score_A,score_B,diff
```

Expected Console Output:

```
joined_rows=0 / id_only_one=2
```

Test Case 3 (Có missing / dòng lỗi)
Input (fileA.csv):

```
A01,8.0
A02,NA
A03,9.5
,7.0
```

Input (fileB.csv):

```
A01,7.5
A03,abc
A04,6.0
```

Expected Output (merged.csv):

```
id,score_A,score_B,diff
A01,8.000000,7.500000,0.500000
```

Expected Console Output:

```
joined_rows=1 / id_only_one=2
```

1.10 Phân tích log file [Nâng cao]

1.10.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `log.txt`, mỗi dòng có dạng:

    `YYYY-MM-DD HH:MM:SS [LEVEL] message`

    với LEVEL $\in$ {`INFO, WARN, ERROR`}
- **Xử lý:**
  - **Case biên:**
    * File rỗng $\rightarrow$ tất cả count = 0, `first_error = NONE`, `last_error = NONE`
    * Dòng sai format $\rightarrow$ bỏ qua
    * Không có dòng ERROR $\rightarrow$ `first_error = NONE`, `last_error = NONE`
  - **Module 1 (Đọc file):**
    * Hàm `readLines(path, vector<string>&)`
    * Đọc toàn bộ dòng bằng `getline`
  - **Module 2 (Parse dòng log):**
    * Hàm `parseLog(line, timestamp, level)`
    * Tách:
      · `timestamp`: 19 ký tự đầu
      · `level`: nằm trong dấu `[ ]`
    * Nếu không đúng format $\rightarrow$ return false
  - **Module 3 (Thống kê):**
    * Đếm số dòng theo từng LEVEL:

      $$\texttt{INFO}, \texttt{WARN}, \texttt{ERROR}$$

    * Nếu gặp ERROR:
      · Nếu là ERROR đầu tiên $\rightarrow$ lưu `first_error`
      · Luôn cập nhật `last_error`
  - **Module 4 (Ghi file):**
    * Hàm `writeSummary(path, info, warn, error, first, last)`
- **Output:**

```
INFO=<n> / WARN=<n> / ERROR=<n> / first_error=<timestamp> /
last_error=<timestamp>
```

1.10.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

// MODULE 1: Read file
bool readLines(string path, vector<string>& lines) {
    ifstream file(path);
    if (!file.is_open()) return false;

    string line;
    while (getline(file, line)) {
        lines.push_back(line);
    }

    file.close();
    return true;
}

// MODULE 2: Parse log line
bool parseLog(const string& line, string& timestamp, string& level) {
    if (line.size() < 22) return false;

    // timestamp = 19 ky tu dau
    timestamp = line.substr(0, 19);

    // tim [LEVEL]
    size_t l = line.find('[');
    size_t r = line.find(']');

    if (l == string::npos || r == string::npos || r <= l + 1) return false;

    level = line.substr(l + 1, r - l - 1);

    if (level != "INFO" && level != "WARN" && level != "ERROR")
        return false;

    return true;
}

// MODULE 3: Analyze
void analyzeLogs(const vector<string>& lines,
                 int& info, int& warn, int& error,
                 string& first_error, string& last_error) {
```

```

        info = warn = error = 0;
        first_error = "NONE";
        last_error = "NONE";

        for (const string& line : lines) {
            string timestamp, level;

            if (!parseLog(line, timestamp, level)) continue;

            if (level == "INFO") info++;
            else if (level == "WARN") warn++;
            else if (level == "ERROR") {
                error++;
                if (first_error == "NONE") first_error = timestamp;
                last_error = timestamp;
            }
        }
    }

    // MODULE 4: Write output
    bool writeSummary(string path,
                      int info, int warn, int error,
                      const string& first_error,
                      const string& last_error) {

        ofstream file(path);
        if (!file.is_open()) return false;

        file << "INFO=" << info
            << " / WARN=" << warn
            << " / ERROR=" << error
            << " / first_error=" << first_error
            << " / last_error=" << last_error;

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<string> lines;

        if (!readLines("log.txt", lines)) {
            cerr << "Khong mo duoc file log.txt\n";
            return 1;
        }

        int info, warn, error;
```

```
        string first_error, last_error;

        analyzeLogs(lines, info, warn, error, first_error, last_error);

        if (!writeSummary("summary.txt", info, warn, error, first_error,
    last_error)) {
            cerr << "Khong ghi duoc file summary.txt\n";
            return 1;
        }

        return 0;
    }
```

1.10.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (log.txt):

```
2026-04-01 10:00:00 [INFO] Start system
2026-04-01 10:01:00 [WARN] Low memory
2026-04-01 10:02:00 [ERROR] Crash detected
2026-04-01 10:03:00 [ERROR] Restart failed
```

Expected Output:

```
INFO=1 / WARN=1 / ERROR=2 / first_error=2026-04-01 10:02:00 / last_error
    =2026-04-01 10:03:00
```

Test Case 2 (Không có ERROR)
Input (log.txt):

```
2026-04-01 10:00:00 [INFO] Start
2026-04-01 10:01:00 [WARN] Something
```

Expected Output:

```
INFO=1 / WARN=1 / ERROR=0 / first_error=NONE / last_error=NONE
```

Test Case 3 (Case biên – file rỗng)
Input (log.txt):

```

```

Expected Output:

```
INFO=0 / WARN=0 / ERROR=0 / first_error=NONE / last_error=NONE
```

2 Tuần 2: Mảng 1D + Đa thức + Thống kê mô tả

2.1 Tìm min, max, mean mảng [Cơ bản]

2.1.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ in `EMPTY`
    * $n = 1 \rightarrow \min = \max = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    * Hàm `readArray(path, vector<double>&)`
    * Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    * Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    * Duyệt mảng 1 lần
- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

2.1.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <fstream>
#include <vector>
```

```
#include <iomanip>

using namespace std;

// MODULE 1: Doc du lieu tu file
bool readArray(string path, vector<double>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file\n";
        return false;
    }

    int n;
    file >> n;

    double x;
    for (int i = 0; i < n; i++) {
        file >> x;
        a.push_back(x);
    }

    file.close();
    return true;
}

// MODULE 2: Tinh mean
double calcMean(const vector<double>& a) {
    if (a.empty()) return 0.0;

    long double sum = 0;
    for (double x : a) sum += x;

    return (double)(sum / a.size());
}

// MODULE 3: Ghi output
bool writeOutput(string path, const vector<double>& a) {
    ofstream file(path);
    if (!file.is_open()) {
        cerr << "Khong ghi duoc file\n";
        return false;
    }

    int n = a.size();

    if (n == 0) {
        file << "EMPTY";
        file.close();
        return true;
```

```
        }

        double mn = a[0], mx = a[0];

        for (int i = 1; i < n; i++) {
            if (a[i] < mn) mn = a[i];
            if (a[i] > mx) mx = a[i];
        }

        double mean = calcMean(a);

        file << fixed << setprecision(4);
        file << "min=" << mn
            << " / max=" << mx
            << " / mean=" << mean;

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<double> a;

        if (!readArray("array.txt", a)) {
            return 1;
        }

        if (!writeOutput("output.txt", a)) {
            return 1;
        }

        return 0;
    }
```

2.1.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)

Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

2.2 Đếm phần tử thỏa điều kiện [Cơ bản]

2.2.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `array.txt` chứa mảng số nguyên: số lượng n và n phần tử.
  - File `config.txt` chứa ngưỡng: một số nguyên k.
- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ mảng rỗng, cả hai biến đếm đều bằng 0.
    * Tất cả phần tử trong mảng đều bằng $k \rightarrow$ `count_ge_k` $= n$, `count_lt_k` $= 0$.
  - **Module 1 (Đọc mảng):**
    * Hàm `readArray(path, vector<int>&)`
    * Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Đọc cấu hình k):**
    * Hàm `readConfig(path, int&)`
    * Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 3 (Đếm theo điều kiện):**
    * Hàm `countIf(vector<int>, k, geq)` với `geq` là biến `bool` để cờ hiệu đếm lớn hơn hoặc bằng (`true`) hay nhỏ hơn (`false`).
  - **Module 4 (Ghi output):**
    * Hàm `writeOutput(path, count_ge_k, count_lt_k)`
- **Output:**

```
count_ge_k=<i> / count_lt_k=<i>
```

2.2.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>

using namespace std;

// MODULE 1: Doc mang tu file
bool readArray(string path, vector<int>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    int n;
    if (file >> n) {
        int x;
        for (int i = 0; i < n; i++) {
            file >> x;
            a.push_back(x);
        }
    }

    file.close();
    return true;
}

// MODULE 2: Doc nguong k tu file config
bool readConfig(string path, int& k) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    file >> k;

    file.close();
    return true;
}

// MODULE 3: Dem so luong phan tu thoa man
int countIf(const vector<int>& a, int k, bool geq) {
    int count = 0;
    for (int x : a) {
        if (geq) {
            if (x >= k) count++;
```

```
            } else {
                if (x < k) count++;
            }
        }
        return count;
    }

    // MODULE 4: Ghi output
    bool writeOutput(string path, int count_ge_k, int count_lt_k) {
        ofstream file(path);
        if (!file.is_open()) {
            cerr << "Khong ghi duoc file " << path << "\n";
            return false;
        }

        file << "count_ge_k=" << count_ge_k
            << " / count_lt_k=" << count_lt_k;

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<int> a;
        int k;

        if (!readArray("array.txt", a)) {
            return 1;
        }

        if (!readConfig("config.txt", k)) {
            return 1;
        }

        int count_ge = countIf(a, k, true);
        int count_lt = countIf(a, k, false);

        if (!writeOutput("output.txt", count_ge, count_lt)) {
            return 1;
        }

        return 0;
    }

```

2.2.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
6 10 -5 8 15 3 8
```

Input (config.txt):

```
8
```

Expected Output (output.txt):

```
count_ge_k=4 / count_lt_k=2
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Input (config.txt):

```
5
```

Expected Output (output.txt):

```
count_ge_k=0 / count_lt_k=0
```

Test Case 3 (Case biên – Tất cả bằng k)
Input (array.txt):

```
4 7 7 7 7
```

Input (config.txt):

```
7
```

Expected Output (output.txt):

```
count_ge_k=4 / count_lt_k=0
```

2.3 Tính đa thức tại nhiều điểm [Cơ bản]

2.3.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `poly.csv` chứa 1 dòng: `degree, a0, a1, ..., an` (phân tách bằng dấu phẩy).
  - File `x_values.csv` chứa nhiều giá trị x, mỗi dòng 1 giá trị.
- **Xử lý:**

- **Case biên:**
  * `degree = 0` → Đa thức là hằng số $P(x) = a_0$.
  * File `x_values.csv` rỗng → File output chỉ in ra đúng dòng header `x,P(x)`.
  * Gặp giá trị khuyết (`NA`, `NaN`, `nan`, hoặc dòng trống) → Bỏ qua không đưa vào tính toán.
- **Module 1 (Đọc hệ số đa thức):**
  * Hàm `readPoly(path, int&, vector<double>&)`
  * Tách chuỗi bằng dấu phẩy (`,`) thông qua `stringstream`.
  * Bắt lỗi mở file (fail → `cerr` + `return false`).
- **Module 2 (Đọc giá trị x):**
  * Hàm `readXValues(path, vector<double>&)`
  * Đọc từng dòng, kiểm tra missing values.
  * Bắt lỗi mở file.
- **Module 3 (Tính giá trị đa thức):**
  * Hàm `evalHorner(vector<double> a, double x)`
  * Áp dụng thuật toán Horner: `res = a[n]; for i=n-1..0: res = res*x + a[i]`.
- **Module 4 (Ghi output):**
  * Hàm `writeOutput(path, vector<double>, vector<double>)`
  * Ghi header `x,P(x)`. Định dạng số thực bằng `fixed` và `setprecision(6)`.

- **Output:** File `output.csv` với định dạng:

```
x,P(x)
<f>,<f>
```

2.3.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include <iomanip>

using namespace std;

// MODULE 1: Doc da thuc tu file poly.csv
bool readPoly(string path, int& degree, vector<double>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
```

```
        return false;
    }

    string line;
    if (getline(file, line)) {
        stringstream ss(line);
        string token;

        // Doc degree
        if (getline(ss, token, ',')) {
            degree = stoi(token);
        }

        // Doc cac he so a0, a1, ..., an
        while (getline(ss, token, ',')) {
            // Kiem tra missing value theo quy dinh (da ngat dong de tranh tran le PDF)
            if (token == "NA" || token == "NaN" ||
                token == "nan" || token.empty()) {
                a.push_back(0.0); // He so khuyet xem nhu bang 0
            } else {
                a.push_back(stod(token));
            }
        }
    }

    file.close();
    return true;
}

// MODULE 2: Doc cac diem x tu x_values.csv
bool readXValues(string path, vector<double>& x_vals) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    string line;
    while (getline(file, line)) {
        // Xu ly gia tri missing / o trong
        if (line.empty() || line == "NA" || line == "NaN" || line == "nan") {
            continue;
        }
        x_vals.push_back(stod(line));
    }

    file.close();
```

```
        return true;
    }

    // MODULE 3: Tinh P(x) bang thuat toan Horner
    double evalHorner(vector<double> a, double x) {
        if (a.empty()) return 0.0;

        int n = a.size() - 1;
        double res = a[n];
        for (int i = n - 1; i >= 0; i--) {
            res = res * x + a[i];
        }
        return res;
    }

    // MODULE 4: Ghi ket qua ra output.csv
    bool writeOutput(string path, const vector<double>& x_vals, const
    vector<double>& a) {
        ofstream file(path);
        if (!file.is_open()) {
            cerr << "Khong ghi duoc file " << path << "\n";
            return false;
        }

        // Ghi dong header
        file << "x,P(x)\n";

        // Ghi du lieu
        file << fixed << setprecision(6);
        for (double x : x_vals) {
            double px = evalHorner(a, x);
            file << x << "," << px << "\n";
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        int degree = 0;
        vector<double> a;
        vector<double> x_vals;

        if (!readPoly("poly.csv", degree, a)) {
            return 1;
        }

        if (!readXValues("x_values.csv", x_vals)) {
```

```
            return 1;
        }

        if (!writeOutput("output.csv", x_vals, a)) {
            return 1;
        }

        return 0;
    }

```

2.3.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường - Bậc 2)
$P(x) = 1 + 2x + 3x^2$ (degree=2)
Input (poly.csv):

```
2,1.0,2.0,3.0
```

Input (x_values.csv):

```
1.5
-2.0
NaN
```

Expected Output (output.csv):

```
x,P(x)
1.500000,10.750000
-2.000000,9.000000
```

Test Case 2 (Case biên – Hàm hằng, degree = 0)
$P(x) = 5.5$
Input (poly.csv):

```
0,5.5
```

Input (x_values.csv):

```
10.0
0.0
```

Expected Output (output.csv):

```
x,P(x)
10.000000,5.500000
0.000000,5.500000
```

Test Case 3 (Case biên – x_values rỗng)
Input (poly.csv):

```
3,1,0,0,5
```

Input (x_values.csv):

```

```

(File trống)

Expected Output (output.csv):

```
x,P(x)
```

2.4 Variance và std deviation [Trung bình]

2.4.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `array.txt` chứa:
    * Số nguyên n (số lượng phần tử).
    * n số thực.
- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ Ghi ra file chữ `EMPTY`.
    * $n = 1 \rightarrow$ variance $= 0$, std $= 0$.
    * Tất cả các phần tử bằng nhau $\rightarrow$ variance $= 0$, std $= 0$.
  - **Module 1 (Đọc mảng):**
    * Hàm `readArray(path, vector<double>&)`
    * Bắt lỗi mở file (fail $\rightarrow$ `cerr` + `return false`).
  - **Module 2 (Tính trung bình - Mean):**
    * Hàm `calcMean(vector<double>)`
    * Dùng kiểu `long double` để cộng dồn tránh sai số.
  - **Module 3 (Tính phương sai và độ lệch chuẩn):**
    * Hàm `calcVariance(vector<double>, double mean)`
    * Tính phương sai quần thể (chia cho n).
    * Tích lũy tổng bình phương độ lệch $\sum(x_i - \text{mean})^2$ bằng `long double` để tránh hiện tượng *catastrophic cancellation* (sai số triệt tiêu).
    * Độ lệch chuẩn (std deviation) $\sigma = \sqrt{\sigma^2}$. Sử dụng hàm `sqrt()` trong thư viện `<cmath>`.
  - **Module 4 (Ghi output):**
    * Hàm `writeOutput(path, vector<double>)`
    * Định dạng số thực bằng `fixed` và `setprecision(6)`.

- **Output:** File `output.txt` với định dạng:

```
mean=<f> / variance=<f> / std=<f>
```

2.4.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <iomanip>
#include <cmath>

using namespace std;

// MODULE 1: Doc mang tu file array.txt
bool readArray(string path, vector<double>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    int n;
    if (file >> n) {
        double x;
        for (int i = 0; i < n; i++) {
            file >> x;
            a.push_back(x);
        }
    }

    file.close();
    return true;
}

// MODULE 2: Tinh mean (trung binh)
double calcMean(const vector<double>& a) {
    if (a.empty()) return 0.0;

    long double sum = 0.0;
    for (double x : a) {
        sum += x;
    }
    return (double)(sum / a.size());
}

// MODULE 3: Tinh variance (phuong sai quan the)
double calcVariance(const vector<double>& a, double mean) {
    if (a.size() <= 1) return 0.0;
```

```

    long double sum_sq_diff = 0.0;
    for (double x : a) {
        double diff = x - mean;
        // Dung long double tich luy de tranh catastrophic cancellation
        sum_sq_diff += (long double)(diff * diff);
    }

    // Phuong sai quan the chia cho n
    return (double)(sum_sq_diff / a.size());
}

// MODULE 4: Ghi ket qua ra output.txt
bool writeOutput(string path, const vector<double>& a) {
    ofstream file(path);
    if (!file.is_open()) {
        cerr << "Khong ghi duoc file " << path << "\n";
        return false;
    }

    if (a.empty()) {
        file << "EMPTY";
        file.close();
        return true;
    }

    double mean = calcMean(a);
    double variance = calcVariance(a, mean);
    double std_dev = sqrt(variance);

    file << fixed << setprecision(6);
    file << "mean=" << mean
        << " / variance=" << variance
        << " / std=" << std_dev;

    file.close();
    return true;
}

// MAIN
int main() {
    vector<double> a;

    if (!readArray("array.txt", a)) {
        return 1;
    }

    if (!writeOutput("output.txt", a)) {
```

```
            return 1;
        }

        return 0;
    }

```

2.4.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.0 20.0 30.0 40.0 50.0
```

Expected Output (output.txt):

```
mean=30.000000 / variance=200.000000 / std=14.142136
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output (output.txt):

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output (output.txt):

```
mean=42.500000 / variance=0.000000 / std=0.000000
```

Test Case 4 (Case biên – Tất cả phần tử bằng nhau)
Input (array.txt):

```
4 7.2 7.2 7.2 7.2
```

Expected Output (output.txt):

```
mean=7.200000 / variance=0.000000 / std=0.000000
```

2.5 Đạo hàm đa thức [Trung bình]

2.5.1 Phân tích & Thiết kế (M0)

- **Input:**

- File `poly.csv` chứa 1 dòng: `degree, a0, a1, ..., an`.
- File `x_values.csv` chứa các giá trị x cần tính đạo hàm (mỗi dòng 1 số).

- **Xử lý:**
  - **Case biên:**
    * `degree = 0` → Đa thức hằng $P(x) = a_0$. Đạo hàm $P'(x) = 0$ (bậc 0, hệ số 0).
    * Giá trị khuyết (`NA`, `NaN`) trong file hệ số → coi là 0. Trong file x → bỏ qua.
  - **Module 1 (Đọc dữ liệu):**
    * Tái sử dụng logic đọc mảng hệ số phân tách bằng dấu phẩy và đọc mảng x từng dòng (đã xử lý missing values).
  - **Module 2 (Tính hệ số đạo hàm):**
    * Hàm `computeDerivCoeffs(vector<double> a)`
    * Áp dụng công thức: `deriv[i] = (i + 1) * a[i + 1]` với i chạy từ 0 đến $n - 1$.
    * Nếu mảng `a` có kích thước ≤ 1 (tương đương `degree = 0`), trả về mảng con chỉ chứa `{0.0}`.
  - **Module 3 (Tính giá trị đa thức):**
    * Tái sử dụng hàm `evalHorner` để tính $P'(x)$ tại các điểm x dựa trên mảng hệ số đạo hàm vừa tìm được.
  - **Module 4 (Ghi outputs):**
    * Hàm `writeDerivative(path, vector<double> deriv)`: Ghi file `derivative.csv` định dạng `degree_deriv, b0, b1...`
    * Hàm `writeOutput(path, vector<double> x_vals, vector<double> deriv)`: Ghi file `output.csv` định dạng `x,P'(x)`.
    * Tuân thủ quy định dùng `fixed` và `setprecision(6)`.

- **Output:**
  - File `derivative.csv`
  - File `output.csv`

2.5.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include <iomanip>

```

```
using namespace std;

// MODULE 1: Doc da thuc tu file poly.csv
bool readPoly(string path, int& degree, vector<double>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    string line;
    if (getline(file, line)) {
        stringstream ss(line);
        string token;

        if (getline(ss, token, ',')) {
            degree = stoi(token);
        }

        while (getline(ss, token, ',')) {
            if (token == "NA" || token == "NaN" ||
                token == "nan" || token.empty()) {
                a.push_back(0.0);
            } else {
                a.push_back(stod(token));
            }
        }
    }
    file.close();
    return true;
}

// MODULE 2: Doc cac diem x tu x_values.csv
bool readXValues(string path, vector<double>& x_vals) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    string line;
    while (getline(file, line)) {
        if (line.empty() || line == "NA" || line == "NaN" || line == "nan") {
            continue;
        }
        x_vals.push_back(stod(line));
    }
    file.close();
```

```
        return true;
    }

    // MODULE 3: Tinh he so da thuc dao ham
    vector<double> computeDerivCoeffs(const vector<double>& a) {
        vector<double> deriv;

        // Case bien: da thuc hang (bac 0) -> dao ham = 0
        if (a.size() <= 1) {
            deriv.push_back(0.0);
            return deriv;
        }

        // Cong thuc: b[i] = (i + 1) * a[i + 1]
        for (size_t i = 0; i < a.size() - 1; i++) {
            deriv.push_back((i + 1) * a[i + 1]);
        }

        return deriv;
    }

    // MODULE 4: Tinh gia tri da thuc bang Horner
    double evalHorner(const vector<double>& a, double x) {
        if (a.empty()) return 0.0;

        int n = a.size() - 1;
        double res = a[n];
        for (int i = n - 1; i >= 0; i--) {
            res = res * x + a[i];
        }
        return res;
    }

    // MODULE 5: Ghi file derivative.csv
    bool writeDerivative(string path, const vector<double>& deriv) {
        ofstream file(path);
        if (!file.is_open()) {
            cerr << "Khong ghi duoc file " << path << "\n";
            return false;
        }

        int degree_deriv = deriv.empty() ? 0 : deriv.size() - 1;
        file << degree_deriv;

        file << fixed << setprecision(6);
        for (double coeff : deriv) {
            file << "," << coeff;
        }
        file << "\n";
```

```
        file.close();
        return true;
    }

    // MODULE 6: Ghi file output.csv
    bool writeOutput(string path, const vector<double>& x_vals, const vector<double>& deriv) {
        ofstream file(path);
        if (!file.is_open()) {
            cerr << "Khong ghi duoc file " << path << "\n";
            return false;
        }

        file << "x,P'(x)\n";
        file << fixed << setprecision(6);

        for (double x : x_vals) {
            double px_prime = evalHorner(deriv, x);
            file << x << "," << px_prime << "\n";
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        int degree = 0;
        vector<double> a;
        vector<double> x_vals;

        if (!readPoly("poly.csv", degree, a)) return 1;
        if (!readXValues("x_values.csv", x_vals)) return 1;

        vector<double> deriv = computeDerivCoeffs(a);

        if (!writeDerivative("derivative.csv", deriv)) return 1;
        if (!writeOutput("output.csv", x_vals, deriv)) return 1;

        return 0;
    }

```

2.5.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường - Bậc 3)
$P(x) = 1 + 2x + 3x^2 + 4x^3 \Rightarrow P'(x) = 2 + 6x + 12x^2$

Input (poly.csv):

```
3,1.0,2.0,3.0,4.0
```

Input (x_values.csv):

```
1.0
-1.0
```

Expected Output (derivative.csv):

```
2,2.000000,6.000000,12.000000
```

Expected Output (output.csv):

```
x,P'(x)
1.000000,20.000000
-1.000000,8.000000
```

Test Case 2 (Case biên – Đa thức hằng, degree = 0)
$P(x) = 5.5 \Rightarrow P'(x) = 0$
Input (poly.csv):

```
0,5.5
```

Input (x_values.csv):

```
10.0
```

Expected Output (derivative.csv):

```
0,0.000000
```

Expected Output (output.csv):

```
x,P'(x)
10.000000,0.000000
```

2.6 Tìm vị trí phần tử lớn nhất [Cơ bản]

2.6.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `array.txt` chứa: số nguyên n và mảng n số nguyên.
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ ghi `EMPTY`.
    - $n = 1 \rightarrow$ `max_val = a[0]`, `max_idx = 0`.
  - **Thuật toán:**

    * Khởi tạo `max_val = a[0]`, `max_idx = 0`.
    * Duyệt mảng từ $i = 1$ đến $n - 1$:
      · Nếu `a[i] > max_val` thì cập nhật.
    * Dùng dấu `>` để đảm bảo khi có nhiều phần tử lớn nhất bằng nhau thì lấy chỉ số nhỏ nhất.
  - **Module 1 (Đọc dữ liệu):** Hàm `readArray`. Bắt lỗi mở file (`return false`).
  - **Module 2 (Xử lý & Ghi kết quả):** Hàm `writeOutput` tìm max và ghi ra file.

- **Output:** File `output.txt`:

```
max_val=<i> / max_idx=<i>
```

2.6.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>

using namespace std;

// MODULE 1: Doc mang tu file
bool readArray(string path, vector<int>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    int n;
    file >> n;

    int x;
    for (int i = 0; i < n; i++) {
        file >> x;
        a.push_back(x);
    }

    file.close();
    return true;
}

// MODULE 2: Tim max va ghi ket qua
bool writeOutput(string path, const vector<int>& a) {
    ofstream file(path);
    if (!file.is_open()) {
```

```
            cerr << "Khong ghi duoc file " << path << "\n";
            return false;
        }

        if (a.empty()) {
            file << "EMPTY";
            file.close();
            return true;
        }

        int max_val = a[0];
        int max_idx = 0;

        for (int i = 1; i < (int)a.size(); i++) {
            if (a[i] > max_val) {
                max_val = a[i];
                max_idx = i+1;
            }
        }

        file << "max_val=" << max_val
            << " / max_idx=" << max_idx;

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<int> a;

        if (!readArray("array.txt", a)) return 1;
        if (!writeOutput("output.txt", a)) return 1;

        return 0;
    }
```

2.6.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 1 5 3 9 2
```

Expected Output (output.txt):

```
max_val=9 / max_idx=4
```

Test Case 2 (Case biên – nhiều phần tử lớn nhất bằng nhau)

Input (array.txt):

```
6 4 8 2 8 1 8
```

Expected Output (output.txt):

```
max_val=8 / max_idx=2
```

Test Case 3 (Case biên – n = 0)
Input (array.txt):

```
0
```

Expected Output (output.txt):

```
EMPTY
```

Test Case 4 (Case biên – n = 1)
Input (array.txt):

```
1 42
```

Expected Output (output.txt):

```
max_val=42 / max_idx=1
```

2.7 Bài 7: Chuẩn hoá mảng Min-Max [Cơ bản]

2.7.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `array.txt` chứa: số nguyên n và mảng n số thực (không có missing).
- **Xử lý:**
  - **Công thức chuẩn hoá:**

$$x' = \frac{x - \min}{\max - \min}$$

  - **Case biên:**
    * $n = 0 \rightarrow$ file output chỉ có header.
    * Nếu $\max = \min \rightarrow$ mọi giá trị `scaled = 0`.
  - **Module 1 (Đọc dữ liệu):** Hàm `readArray`. Bắt lỗi mở file (`return false`).
  - **Module 2 (Tìm min, max):** Duyệt mảng để xác định min và max.
  - **Module 3 (Chuẩn hoá & Ghi kết quả):** Hàm `writeOutput` tính giá trị chuẩn hoá và ghi ra file.

- **Output:** File `normalized.csv`:

```
i,original,scaled
```

2.7.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <iomanip>

using namespace std;

// MODULE 1: Doc mang tu file
bool readArray(string path, vector<double>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    int n;
    file >> n;

    double x;
    for (int i = 0; i < n; i++) {
        file >> x;
        a.push_back(x);
    }

    file.close();
    return true;
}

// MODULE 2 + 3: Chuan hoa va ghi ket qua
bool writeOutput(string path, const vector<double>& a) {
    ofstream file(path);
    if (!file.is_open()) {
        cerr << "Khong ghi duoc file " << path << "\n";
        return false;
    }

    file << "i,original,scaled\n";
    file << fixed << setprecision(6);

    int n = a.size();
    if (n == 0) {
        file.close();
        return true;
```

```
        }

        double mn = a[0], mx = a[0];
        for (double x : a) {
            if (x < mn) mn = x;
            if (x > mx) mx = x;
        }

        for (int i = 0; i < n; i++) {
            double scaled;
            if (mx == mn) scaled = 0.0;
            else scaled = (a[i] - mn) / (mx - mn);

            file << i << "," << a[i] << "," << scaled << "\n";
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<double> a;

        if (!readArray("array.txt", a)) return 1;
        if (!writeOutput("normalized.csv", a)) return 1;

        return 0;
    }
```

2.7.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10 20 30 40 50
```

Expected Output (normalized.csv):

```
i,original,scaled
0,10.000000,0.000000
1,20.000000,0.250000
2,30.000000,0.500000
3,40.000000,0.750000
4,50.000000,1.000000
```

Test Case 2 (Case biên – max = min)
Input (array.txt):

```
3 5 5 5
```

Expected Output (normalized.csv):

```
i,original,scaled
0,5.000000,0.000000
1,5.000000,0.000000
2,5.000000,0.000000
```

Test Case 3 (Case biên – n = 0)
Input (array.txt):

```
0
```

Expected Output (normalized.csv):

```
i,original,scaled
```

2.8 Impute missing bằng median [Trung bình]

2.8.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `data.csv` chứa mảng số thực có missing (`NA`, `NaN`, hoặc dòng trống).
- **Xử lý:**
  - **Quy ước missing:**
    * Dòng trống hoặc `NA`/`NaN` (không phân biệt hoa thường) được xem là missing.
    * Không dùng `-1` làm missing.
  - **Cách tính median:**
    * Lấy các giá trị hợp lệ, sắp xếp tăng dần.
    * Nếu số phần tử hợp lệ lẻ → median là phần tử giữa.
    * Nếu số phần tử hợp lệ chẵn → median là trung bình của 2 phần tử giữa.
    * Nếu không có giá trị hợp lệ nào → median = 0.
  - **Case biên:**
    * File rỗng → `count_valid = 0`, `count_missing = 0`, `median = 0`.
    * Tất cả phần tử đều missing → mọi giá trị impute bằng 0.
  - **Module 1 (Hỗ trợ xử lý chuỗi):**
    * Hàm `trim` để xóa khoảng trắng đầu/cuối.
    * Hàm `toLowerStr` để đưa chuỗi về chữ thường.

    * Hàm `isMissingToken` để kiểm tra missing.
  - **Module 2 (Đọc dữ liệu):** Hàm `loadData`. Đọc từng dòng, lưu lại trạng thái missing và danh sách giá trị hợp lệ. Bắt lỗi mở file (`return false`).
  - **Module 3 (Tính median):** Hàm `getMedian`.
  - **Module 4 (Ghi output):** Hàm `writeOutput` ghi file `output.csv`.
  - **Module 5 (Ghi summary):** Hàm `writeSummary` ghi file `summary.txt`.

- **Output:**
  - File `output.csv`

```
i,value_imputed
```

  - File `summary.txt`

```
count_valid=<i>
count_missing=<i>
median=<f>
```

2.8.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <algorithm>
#include <iomanip>
#include <cctype>

using namespace std;

struct DataNode {
    double value;
    bool isMissing;
};

// MODULE: Trim
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: ToLower
string toLowerStr(string s) {
    for (char& c : s) {
```

```
            c = (char)tolower((unsigned char)c);
        }
        return s;
    }

    // MODULE 1: Kiem tra missing token
    bool isMissingToken(string t) {
        t = trim(t);
        t = toLowerStr(t);

        if (t.empty()) return true;
        return (t == "na" || t == "nan");
    }

    // MODULE 2: Doc du lieu tu file
    bool loadData(string path, vector<DataNode>& data, vector<double>&
    valid) {
        ifstream file(path);
        if (!file.is_open()) {
            cerr << "Khong mo duoc file " << path << "\n";
            return false;
        }

        string line;
        while (getline(file, line)) {
            if (isMissingToken(line)) {
                data.push_back({0.0, true});
            } else {
                double val = stod(trim(line));
                data.push_back({val, false});
                valid.push_back(val);
            }
        }

        file.close();
        return true;
    }

    // MODULE 3: Tinh median
    double getMedian(vector<double> valid) {
        if (valid.empty()) return 0.0;

        sort(valid.begin(), valid.end());
        int n = (int)valid.size();

        if (n % 2 == 1) return valid[n / 2];
        return (valid[n / 2 - 1] + valid[n / 2]) / 2.0;
    }

```

```
// MODULE 4: Ghi output.csv
bool writeOutput(string path, const vector<DataNode>& data, double median) {
    ofstream file(path);
    if (!file.is_open()) {
        cerr << "Khong ghi duoc file " << path << "\n";
        return false;
    }

    file << "i,value_imputed\n";
    file << fixed << setprecision(6);

    for (int i = 0; i < (int)data.size(); i++) {
        double outVal = data[i].isMissing ? median : data[i].value;
        file << i << "," << outVal << "\n";
    }

    file.close();
    return true;
}

// MODULE 5: Ghi summary.txt
bool writeSummary(string path, int countValid, int countMissing, double median) {
    ofstream file(path);
    if (!file.is_open()) {
        cerr << "Khong ghi duoc file " << path << "\n";
        return false;
    }

    file << "count_valid=" << countValid << "\n";
    file << "count_missing=" << countMissing << "\n";
    file << fixed << setprecision(6);
    file << "median=" << median << "\n";

    file.close();
    return true;
}

// MAIN
int main() {
    vector<DataNode> data;
    vector<double> valid;

    if (!loadData("data.csv", data, valid)) return 1;

    double median = getMedian(valid);
    int countValid = (int)valid.size();
    int countMissing = (int)data.size() - countValid;
```

```

        if (!writeOutput("output.csv", data, median)) return 1;
        if (!writeSummary("summary.txt", countValid, countMissing, median)
    ) return 1;

        return 0;
    }
```

2.8.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (data.csv):

```
10
NA
30

20
```

Expected Output (output.csv):

```
i,value_imputed
0,10.000000
1,20.000000
2,30.000000
3,20.000000
4,20.000000
```

Expected Output (summary.txt):

```
count_valid=3
count_missing=2
median=20.000000
```

Test Case 2 (Case biên – tất cả đều missing)
Input (data.csv):

```
NA

NaN
```

Expected Output (output.csv):

```
i,value_imputed
0,0.000000
1,0.000000
2,0.000000
```

Expected Output (summary.txt):

```
count_valid=0
count_missing=3
```

```
median=0.000000
```

Test Case 3 (Case biên – không có missing)
Input (data.csv):

```
5
15
25
35
```

Expected Output (output.csv):

```
i,value_imputed
0,5.000000
1,15.000000
2,25.000000
3,35.000000
```

Expected Output (summary.txt):

```
count_valid=4
count_missing=0
median=20.000000
```

2.9 Histogram đơn giản [Trung bình]

2.9.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `array.txt` chứa: số nguyên n và mảng n số thực.
  - File `config.txt` chứa: số nguyên k là số lượng bin.
- **Xử lý:**
  - **Ý tưởng:**
    * Chia đều khoảng $[\min, \max]$ thành k bin.
    * Độ rộng mỗi bin:
    $$w = \frac{\max - \min}{k}$$
    * Với mỗi phần tử x, chỉ số bin:
    $$idx = \left\lfloor \frac{x - \min}{w} \right\rfloor$$
    * Nếu $x = \max$ thì ép vào bin cuối cùng.
  - **Case biên:**
    * $n = 0$ hoặc $k \leq 0 \rightarrow$ file output rỗng.

    * Nếu max = min → đưa toàn bộ phần tử vào bin đầu tiên, các bin còn lại bằng 0.
  - **Module 1 (Đọc dữ liệu):** Hàm `readArray` và `readK`. Bắt lỗi mở file (`return false`).
  - **Module 2 (Tìm min, max):** Hàm `findRange`.
  - **Module 3 (Đếm bin):** Hàm `countBins`.
  - **Module 4 (Ghi output):** Hàm `writeHistogram` ghi ra file `histogram.txt`.

- **Output:** File `histogram.txt`:

```
[lo, hi): count
```

2.9.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <iomanip>
#include <cmath>

using namespace std;

// MODULE 1a: Doc mang tu file
bool readArray(string path, vector<double>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    int n;
    file >> n;

    double x;
    for (int i = 0; i < n; i++) {
        file >> x;
        a.push_back(x);
    }

    file.close();
    return true;
}

// MODULE 1b: Doc so bin k
bool readK(string path, int& k) {
    ifstream file(path);
    if (!file.is_open()) {
```

```
            cerr << "Khong mo duoc file " << path << "\n";
            return false;
        }

        file >> k;
        file.close();
        return true;
    }

    // MODULE 2: Tim min va max
    void findRange(const vector<double>& a, double& mn, double& mx) {
        mn = mx = a[0];
        for (double x : a) {
            if (x < mn) mn = x;
            if (x > mx) mx = x;
        }
    }

    // MODULE 3: Dem so phan tu trong moi bin
    vector<int> countBins(const vector<double>& a, int k, double mn,
    double mx) {
        vector<int> counts(k, 0);

        if (mx == mn) {
            counts[0] = (int)a.size();
            return counts;
        }

        double width = (mx - mn) / k;

        for (double x : a) {
            int idx = (int)((x - mn) / width);
            if (idx >= k) idx = k - 1;
            counts[idx]++;
        }

        return counts;
    }

    // MODULE 4: Ghi histogram.txt
    bool writeHistogram(string path, int k, double mn, double mx, const
    vector<int>& counts) {
        ofstream file(path);
        if (!file.is_open()) {
            cerr << "Khong ghi duoc file " << path << "\n";
            return false;
        }

        file << fixed << setprecision(2);
```

```

        double width = (mx - mn) / k;
        for (int i = 0; i < k; i++) {
            double lo = mn + i * width;
            double hi = (i == k - 1) ? mx : (mn + (i + 1) * width);
            file << "[" << lo << ", " << hi << "): " << counts[i] << "\n";
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<double> a;
        int k;

        if (!readArray("array.txt", a)) return 1;
        if (!readK("config.txt", k)) return 1;

        ofstream testCreate("histogram.txt");
        if (!testCreate.is_open()) {
            cerr << "Khong ghi duoc file histogram.txt\n";
            return 1;
        }
        testCreate.close();

        if (a.empty() || k <= 0) {
            return 0;
        }

        double mn, mx;
        findRange(a, mn, mx);

        vector<int> counts = countBins(a, k, mn, mx);

        if (!writeHistogram("histogram.txt", k, mn, mx, counts)) return 1;

        return 0;
    }
```

2.9.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 1.0 2.5 3.0 4.8 5.0
```

Input (config.txt):

```
2
```

Expected Output (histogram.txt):

```
[1.00, 3.00): 2
[3.00, 5.00): 3
```

Test Case 2 (Case biên – max = min)
Input (array.txt):

```
4 5 5 5 5
```

Input (config.txt):

```
3
```

Expected Output (histogram.txt):

```
[5.00, 5.00): 4
[5.00, 5.00): 0
[5.00, 5.00): 0
```

Test Case 3 (Case biên – n = 0)
Input (array.txt):

```
0
```

Input (config.txt):

```
4
```

Expected Output (histogram.txt):

```

```

2.10 Nội suy tuyến tính [Nâng cao]

2.10.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `points.csv` chứa các điểm `x,y` đã sắp xếp tăng theo x (không có x trùng).
  - File `queries.csv` chứa các giá trị x_q cần nội suy, mỗi dòng 1 giá trị.
- **Xử lý:**
  - **Nguyên tắc nội suy:**
    - Với mỗi x_q:
      - Nếu $x_q < x_{\min}$ hoặc $x_q > x_{\max}$ → ghi `OUT_OF_RANGE`.

- Nếu x_q trùng đúng một giá trị x_i trong dữ liệu → trả về đúng y_i.
- Nếu $x_i < x_q < x_{i+1}$ → nội suy tuyến tính theo công thức:

$$y_q = y_i + (x_q - x_i) \cdot \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$$

– **Case biên:**

  * File `points.csv` rỗng → chương trình báo lỗi và `return 1`.
  * File `queries.csv` rỗng → file output rỗng.
  * Truy vấn nằm ngoài đoạn $[x_{\min}, x_{\max}]$ → ghi `OUT_OF_RANGE`.

– **Module 1 (Đọc dữ liệu điểm):** Hàm `loadPoints`. Bắt lỗi mở file (`return false`).

– **Module 2 (Nội suy 1 truy vấn):** Hàm `interpolateOne`. Tìm khoảng phù hợp và tính giá trị nội suy.

– **Module 3 (Xử lý toàn bộ truy vấn):** Hàm `processQueries`. Đọc `queries.csv`, xử lý từng dòng và ghi ra `interpolated.csv`.

- **Output:** File `interpolated.csv`:

```
x_q,y_interp
x_q,OUT_OF_RANGE
```

2.10.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include <iomanip>

using namespace std;

struct Point {
    double x;
    double y;
};

// MODULE 1: Doc cac diem tu points.csv
bool loadPoints(string path, vector<Point>& pts) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

```

```
        string line;
        while (getline(file, line)) {
            if (line.empty()) continue;

            stringstream ss(line);
            string sx, sy;

            if (getline(ss, sx, ',') && getline(ss, sy)) {
                Point p;
                p.x = stod(sx);
                p.y = stod(sy);
                pts.push_back(p);
            }
        }

        file.close();
        return true;
    }

    // MODULE 2: Noi suy 1 gia tri xq
    string interpolateOne(const vector<Point>& pts, double xq) {
        if (pts.empty()) return "OUT_OF_RANGE";

        if (xq < pts.front().x || xq > pts.back().x) {
            return "OUT_OF_RANGE";
        }

        for (int i = 0; i < (int)pts.size(); i++) {
            if (xq == pts[i].x) {
                stringstream ss;
                ss << fixed << setprecision(6) << pts[i].y;
                return ss.str();
            }
        }

        for (int i = 0; i < (int)pts.size() - 1; i++) {
            if (pts[i].x < xq && xq < pts[i + 1].x) {
                double xi = pts[i].x;
                double yi = pts[i].y;
                double xj = pts[i + 1].x;
                double yj = pts[i + 1].y;

                double yq = yi + (xq - xi) * (yj - yi) / (xj - xi);

                stringstream ss;
                ss << fixed << setprecision(6) << yq;
                return ss.str();
            }
        }
```

```

        return "OUT_OF_RANGE";
    }

    // MODULE 3: Xu ly danh sach truy van va ghi file
    bool processQueries(string queryPath, string outPath, const vector<Point>& pts) {
        ifstream fin(queryPath);
        if (!fin.is_open()) {
            cerr << "Khong mo duoc file " << queryPath << "\n";
            return false;
        }

        ofstream fout(outPath);
        if (!fout.is_open()) {
            cerr << "Khong ghi duoc file " << outPath << "\n";
            return false;
        }

        fout << fixed << setprecision(6);

        string line;
        while (getline(fin, line)) {
            if (line.empty()) continue;

            double xq = stod(line);
            string ans = interpolateOne(pts, xq);

            fout << xq << "," << ans << "\n";
        }

        fin.close();
        fout.close();
        return true;
    }

    // MAIN
    int main() {
        vector<Point> pts;

        if (!loadPoints("points.csv", pts)) return 1;
        if (pts.empty()) {
            cerr << "File points.csv rong\n";
            return 1;
        }

        if (!processQueries("queries.csv", "interpolated.csv", pts)) return 1;

```

```
        return 0;
    }
```

2.10.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (points.csv):

```
1,2
3,6
5,10
```

Input (queries.csv):

```
2
4
```

Expected Output (interpolated.csv):

```
2.000000,4.000000
4.000000,8.000000
```

Test Case 2 (Case biên – truy vấn trùng điểm có sẵn)
Input (points.csv):

```
0,1
2,5
4,9
```

Input (queries.csv):

```
2
4
```

Expected Output (interpolated.csv):

```
2.000000,5.000000
4.000000,9.000000
```

Test Case 3 (Case biên – ngoài khoảng)
Input (points.csv):

```
1,2
3,6
5,10
```

Input (queries.csv):

```
0
6
```

Expected Output (interpolated.csv):

```
0.000000,OUT_OF_RANGE
6.000000,OUT_OF_RANGE
```

Test Case 4 (Case biên – queries.csv rỗng)
Input (points.csv):

```
1,2
3,6
5,10
```

Input (queries.csv):

```

```

Expected Output (interpolated.csv):

```

```